



Informe Preliminar del Desafío I: Implementación de Sweet Crush mediante manipulación de bits

Estudiante: [Antonio Carlos Merlano Ricardo - 1010152199]

Informática II **Semestre:** 2026-2

Fecha: 11 de septiembre de 2026

Análisis del problema

El proyecto consiste en el diseño e implementación del juego Sweet Crush en C++ utilizando Qt. La restricción principal del sistema es el manejo de la memoria: el tablero debe almacenarse en un único bloque de memoria dinámica (unsigned char*), donde cada ficha ocupe exactamente 3 bits.

Esto significa que no se pueden usar estructuras de almacenamiento ya hechas ni crear un tipo de dato para cada posición. En su lugar, se debe trabajar directamente con la memoria utilizando operadores de bits como &, |, <<, >>, ~ y ^. Por eso, una de las partes más importantes del desarrollo será separar la forma en que se guardan y organizan los datos de las reglas y funcionamiento del juego, para que el programa no se vuelva demasiado complicado.

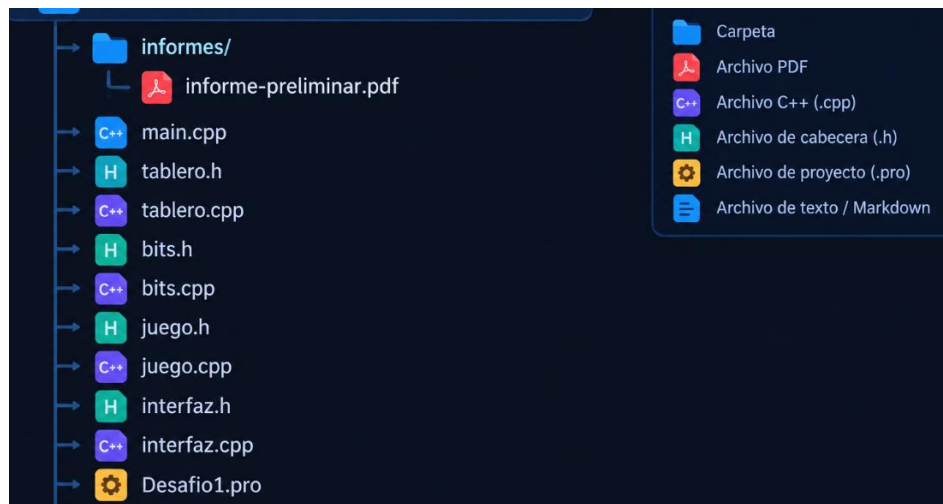
Esquema de tareas y diseño modular

La solución está en un enfoque de diseño modular, donde se dividirá el sistema según la responsabilidad específica de cada componente para garantizar un código escalable y depurable:

1. Módulo bits (.h y .cpp): Se encargará de guardar y manejar la información de las fichas utilizando bits. Aquí se realizarán las operaciones necesarias para leer y modificar los datos.
2. Módulo tablero (.h y .cpp): Se va a encargar de crear y organizar el tablero del juego. También permitirá agregar o eliminar filas y columnas según lo que ocurra durante la partida.

3. Módulo juego (.h y .cpp): Es el que tendrá las reglas principales del juego. Será el encargado de revisar las combinaciones, hacer que las fichas bajen, llenar los espacios vacíos y llevar el conteo de puntos y movimientos.

4. Módulo interfaz (.h y .cpp): Lo usaremos para mostrar el juego al usuario. Permitirá interactuar con el programa y mostrará gráficamente el tablero, las fichas y la información de la partida.



(Estructura modular preliminar diseñada)

Representación de datos y memoria dinámica

Con 3 bits se disponen de 8 estados posibles. Mi idea es que se asignarán seis códigos binarios (ej. 000 a 101) para definir los tipos de fichas del juego. Además pienso que los dos códigos restantes actuarán como estados de control interno: un estado para posiciones "vacías" (110), necesario durante la reorganización del tablero, y un estado de "marca" (111) para identificar fichas pertenecientes a una combinación antes de su eliminación definitiva.

La reserva inicial de memoria será estrictamente exacta a las dimensiones lógicas ingresadas, evitando bloques sobredimensionados.

Evolución de la solución: Lógica y cascadas (PREELIMINAR)

Para encontrar las combinaciones, el programa revisará el tablero buscando tres o más fichas iguales que estén juntas, ya sea de forma horizontal o vertical.

Cuando una ficha haga parte de más de una combinación, el programa la identificará antes de eliminar las fichas. De esta manera, todas las fichas que formen parte de una combinación se eliminan al mismo tiempo.

Después de eliminar las fichas, las que estén por encima pienso que se pueden bajar para ocupar los espacios vacíos. Cuando este movimiento genere una nueva combinación, se contará como una cascada.

Finalmente, los espacios que queden vacíos se pueden llenar con nuevas fichas de manera aleatoria. El proceso continuará automáticamente hasta que ya no existan nuevas combinaciones.

Resumen:

Buscar combinaciones → eliminar fichas → hacerlas caer → rellenar espacios → revisar nuevamente → repetir si hay otra combinación.

Modificación estructural y Regla del 65%

Eliminar una fila o columna → revisar cuánto espacio se está utilizando → si el nuevo tamaño ocupa menos del 65% de la memoria actual, reducir el espacio reservado → si ocupa más del 65%, mantener la memoria existente para evitar cambios innecesarios → finalmente, reorganizar los datos y dejar en cero los espacios que ya no se utilizan.

Problemas que puedo tener al programar o desarrollos anticipados

- **Fichas partidas entre bytes:** Como usamos 3 bits, muchas fichas van a quedar cortadas entre dos bytes distintos. Hacer las máscaras para sacar esos pedacitos sin dañar los bits vecinos será lo más difícil del código.
- **Descuadre del tablero:** Si borro una columna del medio, todo se corre de lugar. Me tocará extraer y volver a empacar la memoria ficha por ficha; un solo error en la matemática y se me desarma el tablero.

- **Fugas de memoria:** Como el tablero cambia de tamaño seguido, usaré mucho `new` y `delete[]`. Tendré que ser cuidadoso para no dejar basura en la RAM y evitar que el juego se cierre de la nada.